Minard for Operations — provision / build / process as one surface

2026-07-19. Design session following Levantine W1–W4 (the proof-of-concept AFC accepted). Status: DESIGN — phases 2–3 actionable now, phase 4 pending the spikes.

0. The brief (AFC, verbatim in spirit)

Build the Minard of provision/build/process-control — in a sense, Minard-for-ShapedSteer. The aim is the feeling Minard gives to a genuinely enormous codebase: **nothing is hidden, yet all is comprehensible.** Three design laws carry over from Minard even though the views themselves will differ:

1. **Tufte: "to clarify, add detail."** Hiding things is where complexity comes from; people imagine they are simplifying when they hide. No rollups-by-default; every unit of the operational world is a mark somewhere.
2. **Modern screens are enormous.** Density is a budget to spend, not a danger to avoid — before animation or highlighting is even considered. The whole fleet fits on one screen as marks.
3. **Semantic zoom** keeps the picture coherent as the user drills into detail. The concepts transfer; the specific views need not.
4. **Totality ⇒ coherence** (AFC, 2026-07-19, "previously unstated rubric"): by designing systems that can show EVERYTHING, we get coherence for free when we show slices or projections — every view is a projection of one total model, so views cannot disagree. The ambition this must survive: n versions of node/npm, m of PureScript, o of Rust all coexisting-but-siloed in a Nix store; version control; builds; processes *including servers inside the BEAM and processes inside containers*; across potentially thousands of machines. Coherence at that scale is not a nicety — it is the product.

The plan: (1) this design; (2) identify required functionality in the underlying systems — quartermaster, bosun, Nix/Nyx, warrant/JTMS; (3) develop and test the gaps in the Levantine prototype; (4) switch gears and build the real thing.

1. The unifying claim: three strata, one epistemic shape

Provision, build, and process-control are the same *kind* of thing. Each has an **intent store** (what should be), an **observable world** (what is), and a **derivation** relating them (is the world as the intent demands, and why). This is exactly warrant's shape — and ShapedSteer's: a typed DAG of computations brought to life by executors. The pattern is already instantiated three times in the ecosystem, separately:

Stratum	Intent store	Observable world	Derivation	Today's owner
Provision	flake.nix + flake.lock	/nix/store paths, profiles, host capabilities	"is provisioned"	Quartermaster (flake + qm-nix.sh; verify/build/publish typed verbs)

Stratum	Intent store	Observable world	Derivation	Today's owner
Build	warrant journal (CanMake facts)	files + mtimes	"is fresh" (Stale/Drifted/Blocked…)	warrant (+ Levantine surface)
Process	compose.yml (typed IR: deps, probes, restart policy)	pids, ports, probe results	"is healthy-running"	Bosun (serve/supervise) + Chair

The unification is not cosmetic. Two proofs:

- **Process staleness is warrant staleness.** A running process whose binary or config was rebuilt after it started is stale in exactly the mtime sense: `mtime(artifact) > startTime(process)`. DeepStar's `verify` ("is the running code the deployed code?" — identity and staleness checks) is literally `still?` asked of a process. The rig doctor independently invented warrant's verb.
- **Bosun's compose is already a fact family.** Typed dependency edges with a requirement gradient (`binds-to`, `part-of`, `requires-ready`), typed health probes, restart policies — this is knowledge about derivability and demandedness, in warrant's sense, wearing different field names. An importer (the Makefile-importer move, reapplied) turns compose into journal facts without Bosun changing at all.

So: **one belief calculus (warrant/JTMS), three observation domains, one reading surface.** The state algebra generalizes cleanly:

Warrant state	Provision reading	Process reading
Missing	not installed	down
Stale	version behind the lock	running but artifact/config newer than start
Fresh	provisioned, matches manifest	healthy, probes green, artifact older than start
Drifted	manifest no longer matches flake (intent changed)	running under a superseded spec
Blocked	gated (e.g. requires signing key present)	held (`bosun supervise --held`; raised deliberately)
WillRun / Running	realizing now	starting (boot grace)

Plus **one genuinely new state the fleet forces on us**:

- **Unobserved.** Warrant's engine is total over an *observed* snapshot — Exists xor Missing, asserted never inferred. A remote machine that doesn't answer ssh is neither. qm-nix.sh already learned this the hard way (reachability-first, own exit code: "could not observe must never report as a world-fact"). The surface must render it honestly: not red, not green — *gray, with the age of the last*

observation. This is a first-class epistemic state the JTMS layer needs, not a UI nicety. It also covers the disconnected-browser case (live page whose server died).

The vertical payoff. Cross-strata edges are what no current tool shows: a listening port depends on a binary depends on a toolchain. "Why is calypso down?" should trace in one proof: process held ← binary drifted ← recipe superseded ← toolchain fine (provisioned 2026-07-19, manifest signed substrate-1). One TidyDag from `/nix/store` to a socket.

2. The surface

Concepts, not final views — per AFC, the views won't be Minard's but the concepts will.

2.1 Semantic zoom as a discrete ladder (the Minard lesson)

Minard's working semantic zoom is a **discrete scale ladder** (package-set → package → module → declaration) with drill-down; its animated ScaleTransition engine exists in Types.purs but was never wired. Lesson: ship the ladder, treat animated transitions as polish. The operations ladder:

```
fleet  →  machine  →  stratum/group  →  unit  →  proof
(all      (its three   (a supervise     (one     (the why card:
machines)  strata)      group, a         process, plain register,
                        journal, a       target,  engine register
                        flake shell)     store    behind the fold)
                                         path)
```

The why card is the innermost zoom level. Levantine's two-register explanation is semantic zoom applied to *prose*: the plain sentence is the zoomed-out reading of the derivation, the engine lines are the zoomed-in one, and the fold is the zoom gesture. This is the deepest sense in which Levantine was the dry-run.

2.2 Density and overlays (the ColorMode move)

Everything is a mark: ~50 processes, hundreds of build targets, thousands of store paths — well inside one screen's budget, exactly as Minard renders thousands of declarations. Extra dimensions arrive as **overlay color modes** (Minard's `ColorMode` pattern — orthogonal channels, not more marks):

- **State** (the traffic lights — default)
- **Provenance** (asserted-by-human / imported-from / generated-by — the agent-ramp channel)
- **Observation age** (how stale is our *knowledge*, distinct from how stale the *world* is — the Unobserved gradient)
- **Churn** (restart counts, rebuild frequency — heat)
- **Machine** (in fleet views)

2.3 The delta as geometry (ghost rendering)

AFC's parked idea, promoted to a core concept: render the difference between the world and the demanded world as **geometry, not just color**. A stale artifact = the existing thing displaced aside + a dotted ghost in tree position; the updated source flows to the ghost; displacement cascades up the tree.

Generalization: **a plan is ghosts before anything runs** — "what would build/provision/restart do?" renders as the ghost layer, then execution animates ghosts becoming solid. Dry-run as a picture. (TidyDag post-processing pass.)

2.4 One live transport

Nothing in the ecosystem streams today except warrant's Live protocol (built this week): Bosun's `/state` is poll-only (Chair polls at 1.5 s), Quartermaster's manifests are static files. Rather than demand event feeds of every system, the design imposes an **observation bus**: per-domain observers either receive native events (fs.watch) or poll-and-diff (bosun `/state`, ssh probes) and emit the same event vocabulary warrant's Live protocol already defines (Reobserved / Resaturated / Running / Finished / Done + Unobserved). The cartography server is a composition of observers; the browser keeps the belief core in-page (the Levantine split, proven in W4).

3. Required functionality in the underlying systems (phase 2)

Ordered by leverage; ★ = small and unblocking.

warrant / JTMS

- ★ **Unobserved as a first-class state.** Snapshot must distinguish "observed absent" from "not observed"; the engine's totality discipline extends (Exists xor Missing xor Unobserved, all asserted). Explain gets honest prose ("I have not seen this machine since 14:02").
- **Observers as data.** Today observation = fs mtimes. Generalize the Snapshot source: an observation domain is (paths ⊂ namespace, observer, clock). Process observer: pid alive / port listening / startedAt / probe status. Provision observer: store-path presence, profile contents. Same snapshot algebra.
- **Importers for compose.yml and flake manifests** — the Makefile importer move. Compose's typed edges/probes and the flake's lock+manifest become journal facts with `ImportedFrom` provenance. Bosun and Quartermaster change *nothing*.
- **A shared namespace** so cross-strata edges resolve: process unit ids, artifact paths, store paths in one Path discipline (probably URI-ish prefixes: `proc:calypso-node`, `nix:/nix/store/…`, plain paths for files).
- **Manifests / tree-shaped outputs** (Pappardelle item (c)) — needed for store-path cones and multi-file targets; NAR digests give content identity for cross-machine "same artifact?" claims.

bosun

- ★ **`startedAt` in `/state`.** The single field that unblocks the process-staleness derivation (`mtime(artifact) > startedAt`). Today `/state` has up/pid/probe/restarts but no wall-clock start.
- **Nothing else, initially.** Poll-and-diff adapts `/state` to the bus; compose is imported, not queried. (A native event feed is a later nicety.)
- **Nested observers, later but load-bearing**: the process stratum recurses — a BEAM node contains supervised processes (bosun's deferred `BEAM-OBSERVER.md` anticipated exactly this), a container contains a process tree. The observer abstraction must compose: an observer can yield units that are themselves observable worlds. This is the semantic-zoom ladder appearing in the *observation* layer, which is reassuring rather than accidental.

quartermaster

- `quartermaster provision` **as a typed verb** (fold qm-nix.sh in, per the existing parking-lot item), whose *output is journal entries*: provisioned-facts with provenance, host, timestamp — the manifests stop being static txt dumps and become assertions.
- ★ **Reachability honesty** already exists (exit 3); it maps directly onto Unobserved.

Nix / Nyx

- Nothing new for v1 beyond what the mandate already gives: the flake defines provision-intent; store paths are observable; NAR digests when manifests land. Nyx stays someday — but note the cartography makes Nyx more valuable (Nix expressions as facts a PureScript toolchain can produce and reason about).

hylograph

- **Semantic-zoom ladder machinery as a library** — the thing Minard wanted (its unwired ScaleTransition is the appetite made visible). Discrete levels, per-level renderers over one model, camera pan/zoom (exists) + level switching; animated transitions later. Build it FOR the ops surface, design it so Minard can adopt it.
- **TidyDag extensions**: ghost/delta post-processing pass; stage banding; forest-of-forests (machine × stratum grouping).

the bus / server

- Generalize warrant-server: N observers (fs.watch, bosun-poll, ssh-probe) → one event stream → many subscribers. The W4 architecture is the seed; this is growth, not replacement.

4. Levantine prototype: gaps to develop and test (phase 3)

Each spike answers a design question before phase 4 commits to it.

1. **fs.watch push** — server watches the build root, broadcasts Reobserved on change; lights move as files are saved. *Question answered: does the observation bus feel right with zero-latency observation?* (Smallest, highest joy.)
2. **Unobserved end-to-end** — kill the server mid-session: the page must gray its lights and say since-when, not keep asserting a stale world. *Question: what does honest disconnection look like, and what does the JTMS need for it?*
3. **Process-observer spike** — poll bosun `/state` (or a fake), emit process facts into the same tree as file facts; derive process-staleness from startedAt vs artifact mtime. *Question: does the state algebra genuinely carry the third stratum, or does process-ness leak into the engine?*
4. **Cross-strata edge** — one tree where `proc:demo` depends on `public/bundle.js` depends on (stubbed) `nix:purs`. The vertical why: "down because the binary is stale because you edited Main." *Question: does the shared namespace work; does the why read naturally across strata?*
5. **Semantic-zoom spike** — a big model (api-index-scale or synthetic): collapse-by-stage at low zoom, unit level at high, why card innermost. *Question: what does the hylograph ladder library need to be?*
6. **Ghost/plan rendering spike** — the displaced-current + dotted-needed pass over TidyDag, driven first by ordinary staleness, then as what-would-build-do. *Question: is the delta geometry legible at density?*

Suggested order: 1, 2, 3, 4 (they compound), then 5 and 6 (view research, parallelizable with early phase 4).

5. Phase 4 shape — Brunel, by assembly (decided 2026-07-19)

Name: Brunel (GitHub clash fallback: Isambard).

Strategy (AFC): build Minard-for-Nix separately first, then assemble — or rewrite — Minard-for-Nix + Bosun's Chair + Levantine into the single webapp. Three proven single-stratum surfaces, one assembly. This answers the Chair-subsumption question: the Chair is absorbed at assembly time, not before.

- **Minard-for-Nix** (the new piece, buildable now): the provision stratum's own cartography. The Nix store is already a content-addressed Merkle DAG — the references graph, closures, GC roots, profiles and their generations, flake inputs. The signature picture is the SILO view AFC described: n versions of node, m of purescript, o of rust coexisting without interference — visible as disjoint closure cones sharing only what they truly share. Also the replication picture: two machines' realized manifests as overlapping marks (identical = the substrate working). hylograph- graph territory; TidyDag or containment layouts over real `nix path-info -r` / `nix why-depends` data.
  - **The Nyx REPL** (AFC, 2026-07-19): conceptually, a REPL whose evaluations show as LIVE CHANGES on the store cartography — evaluate an expression, watch the closure cone it demands appear (ghosts for what would be realized, solid as realization lands). Laziness, sharing, and closure growth — the genuinely hard-to-grasp parts of Nix — become visible mechanics. Pairs Nyx (Marginalia 259) with Minard-for-Nix as mutually motivating.
- **Architecture of the assembly**: the Levantine split scaled up. Browser: Halogen + HATS, warrant core in-page, federated belief stores spanning strata, ladder + overlays + ghosts. Server: the observation bus (observers as data; local fs + bosun poll + ssh probes + nix store reader), journal custody, execution (build recipes; process control via bosun's /control — the surface *asks* bosun, never spawns).
- **Authority boundaries preserved**: bosun keeps process authority, quartermaster keeps provisioning authority, warrant keeps build execution. Brunel *reads everything and commands through the owners* — Minard's read-only ethos, with control gestures delegated (and gated by belief premises: `restart requires approved:by-human` is the same approval-gate primitive as W5).
- **Levantine's fate**: its live/expert surface is assembled into Brunel; the novice teaching surface (templates, sandbox, the onboarding story) survives as the on-ramp — possibly as Brunel's beginner mode, possibly standalone. Decide at assembly.

5.1 ShapedSteer repositioning (AFC, 2026-07-19)

The earlier ShapedSteer framing — merging Spreadsheets, Notebooks and Build Systems in one workbench — is superseded. Where we've arrived: **shared infrastructure** (the JTMS as the belief calculus, Hylograph as the shared graphical language) under **two apps**: a Spreadsheet/Notebook surface (the remaining ShapedSteer workbench idea), and Brunel (build systems belong with process and provisioning). ShapedSteer's L2 Build-à-la-Carte layer and L4 DAG core remain the conceptual ancestors; warrant is the belief-bearing descendant.

6. Questions — answered 2026-07-19 except (5)

1. **Name: Brunel** (AFC). Fallback on a GitHub clash: **Isambard**.
2. **Repo home**: under ShapedSteer/ or CodeExplorer/ — either acceptable (AFC). Leaning ShapedSteer for sibling-hood with bosun and quartermaster; decide at repo creation.
3. **Journals: federated** (AFC confirmed) — per-domain journals of facts/events with a federating reader, matching ownership.
4. **Chair subsumption**: answered by the assembly strategy (§5) — Minard-for-Nix first, then Brunel assembles/rewrites Minard-for-Nix + Chair + Levantine.
5. **History scrubber** in v1 or after? Still open. (The journal makes it cheap in principle; the observation bus makes world-history the actual cost.)

7. Cross-references

- Levantine design: `levantine/docs/DESIGN.md` (incl. parking lot: ghost rendering, Minard-family framing)
- Substrate vision: `kb/architecture/substrate-vision.md`; Nix base layer: `kb/plans/nix-base-layer.md`
- Bosun IR & control: `ShapedSteer/bosun/docs/{OVERVIEW,GRAPH-GRAMMAR,CONTROL-SURFACE,PORT-MODEL,PROVISIONING-SEAM}.md`
- Warrant live protocol: `warrant/src/Warrant/Live/Protocol.purs`
- Minard scale model: `CodeExplorer/minard/docs/transition-matrix.md`